

ASSIGNMENT-12.5

NAME: SRAVANI

ROLLNO: 2303A510G7

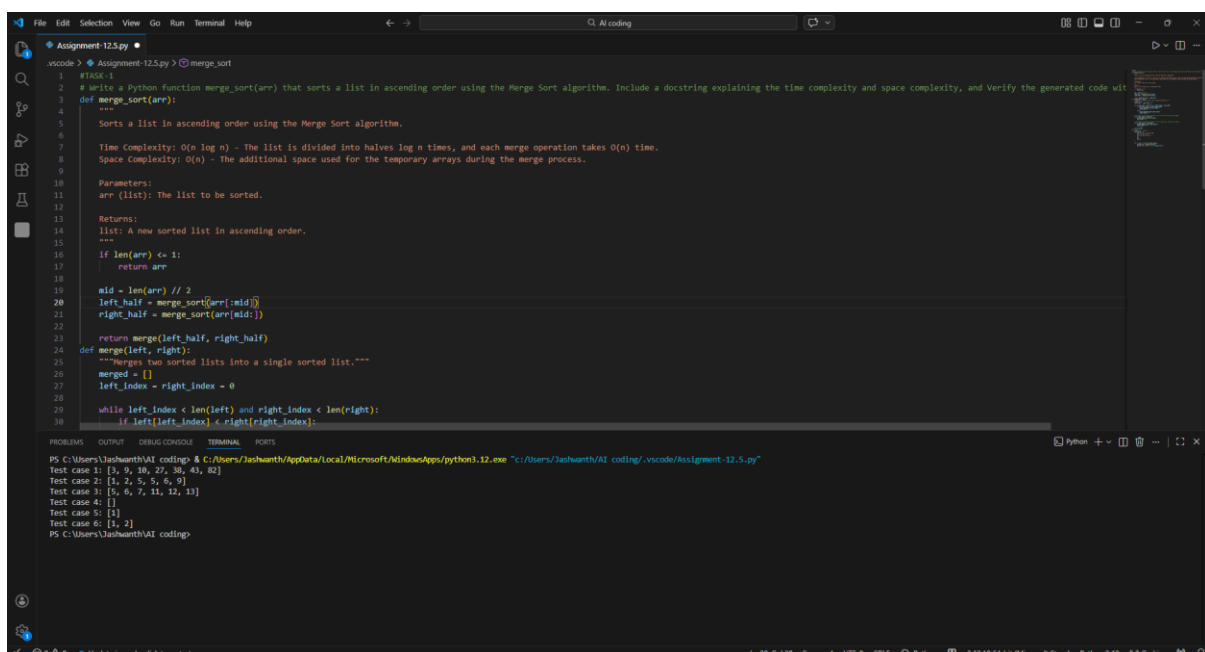
BATCH: 30

TASK-1

PROMPT:

Write a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include a docstring explaining the time complexity and space complexity, and Verify the generated code with test cases.

CODE:



```
1 #task-1
2 # Write a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm. Include a docstring explaining the time complexity and space complexity, and Verify the generated code with test cases.
3 def merge_sort(arr):
4     """
5     Sorts a list in ascending order using the Merge Sort algorithm.
6
7     Time Complexity: O(n log n) - The list is divided into halves log n times, and each merge operation takes O(n) time.
8     Space Complexity: O(n) - The additional space used for the temporary arrays during the merge process.
9
10    Parameters:
11    arr (list): The list to be sorted.
12
13    Returns:
14    list: A new sorted list in ascending order.
15    """
16    if len(arr) <= 1:
17        return arr
18
19    mid = len(arr) // 2
20    left_half = merge_sort(arr[:mid])
21    right_half = merge_sort(arr[mid:])
22
23    return merge(left_half, right_half)
24
25 def merge(left, right):
26     """Merges two sorted lists into a single sorted list."""
27     merged = []
28     left_index = 0
29     right_index = 0
30     while left_index < len(left) and right_index < len(right):
31         if left[left_index] < right[right_index]:
32             merged.append(left[left_index])
33             left_index += 1
34         else:
35             merged.append(right[right_index])
36             right_index += 1
37     merged.extend(left[left_index:])
38     merged.extend(right[right_index:])
39     return merged
40
41 # Test cases
42 test_cases = [
43     [3, 9, 18, 27, 36, 45, 82],
44     [1, 2, 5, 5, 6, 9],
45     [5, 6, 7, 11, 12, 13],
46     [],
47     [1],
48     [4, 2]
49 ]
50
51 for i, test_case in enumerate(test_cases):
52     sorted_list = merge_sort(test_case)
53     print(f"Test case {i+1}: {test_case} -> {sorted_list}")
```

OBSERVATION:

In this task, was to implement the **Merge Sort algorithm** to sort a list of numbers in ascending order.

A prompt was given to the AI asking it to generate a Python function named **merge_sort(arr)** and include a docstring explaining the **time complexity and space complexity** of the algorithm.

The AI generated code that divides the list into two halves recursively and then merges the sorted halves using a helper merge function. Test cases were executed to verify the correctness of the algorithm.

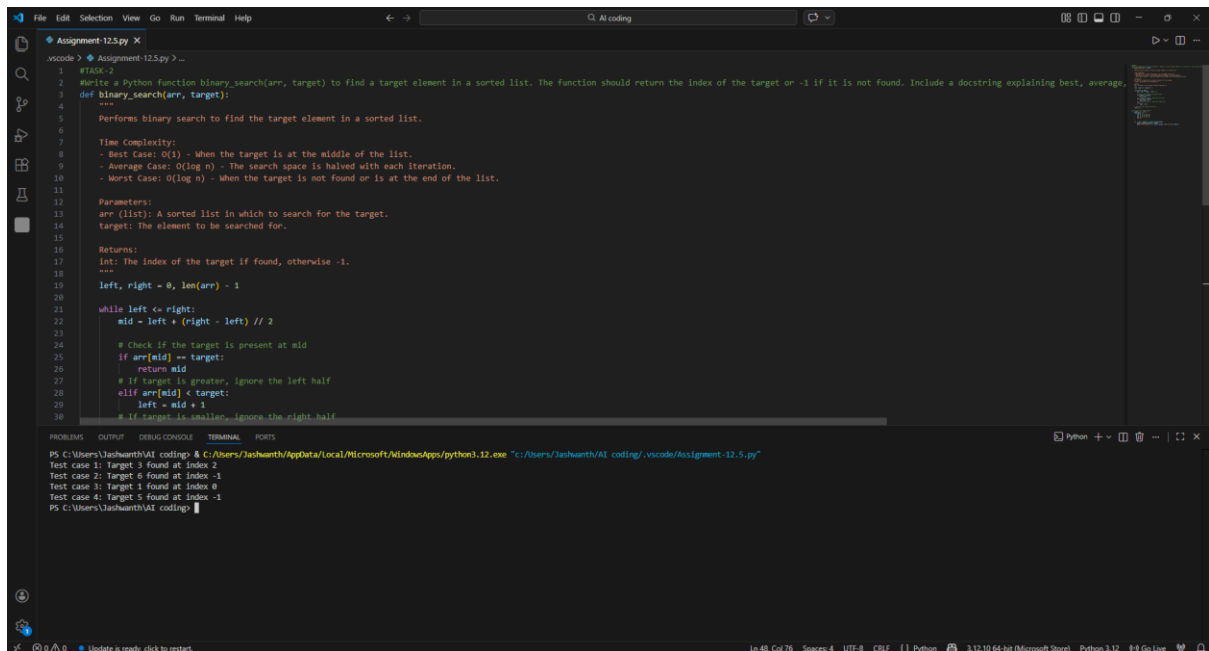
From the output, it was observed that the list was successfully sorted. Merge Sort showed consistent performance with **$O(n \log n)$** time complexity.

TASK-2:

PROMPT:

#Write a Python function `binary_search(arr, target)` to find a target element in a sorted list. The function should return the index of the target or -1 if it is not found. Include a docstring explaining best, average, and worst-case time complexity and Test with various inputs.

CODE:



```
1 #Task-2
2 #Write a Python function binary_search(arr, target) to find a target element in a sorted list. The function should return the index of the target or -1 if it is not found. Include a docstring explaining best, average,
3 def binary_search(arr, target):
4     """
5     Performs binary search to find the target element in a sorted list.
6
7     Time Complexity:
8     - Best Case: O(1) - When the target is at the middle of the list.
9     - Average Case: O(log n) - The search space is halved with each iteration.
10    - Worst Case: O(log n) - When the target is not found or is at the end of the list.
11
12    Parameters:
13    arr (list): A sorted list in which to search for the target.
14    target: The element to be searched for.
15
16    Returns:
17    int: The index of the target if found, otherwise -1.
18    """
19    left, right = 0, len(arr) - 1
20
21    while left <= right:
22        mid = left + (right - left) // 2
23
24        # Check if the target is present at mid
25        if arr[mid] == target:
26            return mid
27        # If target is greater, ignore the left half
28        elif arr[mid] < target:
29            left = mid + 1
30        # If target is smaller, ignore the right half
31
32    return -1
33
34 # Test cases
35 test_cases = [
36     (1, 1),
37     (2, 2),
38     (3, 3),
39     (4, 4),
40     (5, 5),
41     (6, 6),
42     (7, 7),
43     (8, 8),
44     (9, 9),
45     (10, 10),
46     (11, -1),
47     (12, -1),
48     (13, -1),
49     (14, -1),
50     (15, -1),
51 ]
52
53 for i, (target, expected) in enumerate(test_cases):
54     result = binary_search(arr, target)
55     print(f"Test case {i+1}: Target {target} found at index {result} (Expected: {expected})")
56
57 # Output:
58 # Test case 1: Target 1 found at index 0 (Expected: 0)
59 # Test case 2: Target 2 found at index 1 (Expected: 1)
60 # Test case 3: Target 3 found at index 2 (Expected: 2)
61 # Test case 4: Target 4 found at index 3 (Expected: 3)
62 # Test case 5: Target 5 found at index 4 (Expected: 4)
63 # Test case 6: Target 6 found at index 5 (Expected: 5)
64 # Test case 7: Target 7 found at index 6 (Expected: 6)
65 # Test case 8: Target 8 found at index 7 (Expected: 7)
66 # Test case 9: Target 9 found at index 8 (Expected: 8)
67 # Test case 10: Target 10 found at index 9 (Expected: 9)
68 # Test case 11: Target 11 not found at index -1 (Expected: -1)
69 # Test case 12: Target 12 not found at index -1 (Expected: -1)
70 # Test case 13: Target 13 not found at index -1 (Expected: -1)
71 # Test case 14: Target 14 not found at index -1 (Expected: -1)
72 # Test case 15: Target 15 not found at index -1 (Expected: -1)
```

OBSERVATION:

In this task, was to implement the **Binary Search algorithm** to find a target element in a sorted list.

A prompt was provided to the AI asking it to generate a function called **`binary_search(arr, target)`** that returns the index of the target element or **-1 if the element is not found**. The prompt also requested explanations of **best, average, and worst case time complexities**.

The generated code repeatedly divides the search space in half until the element is found or the search range becomes empty. Different test inputs were used to verify the functionality.

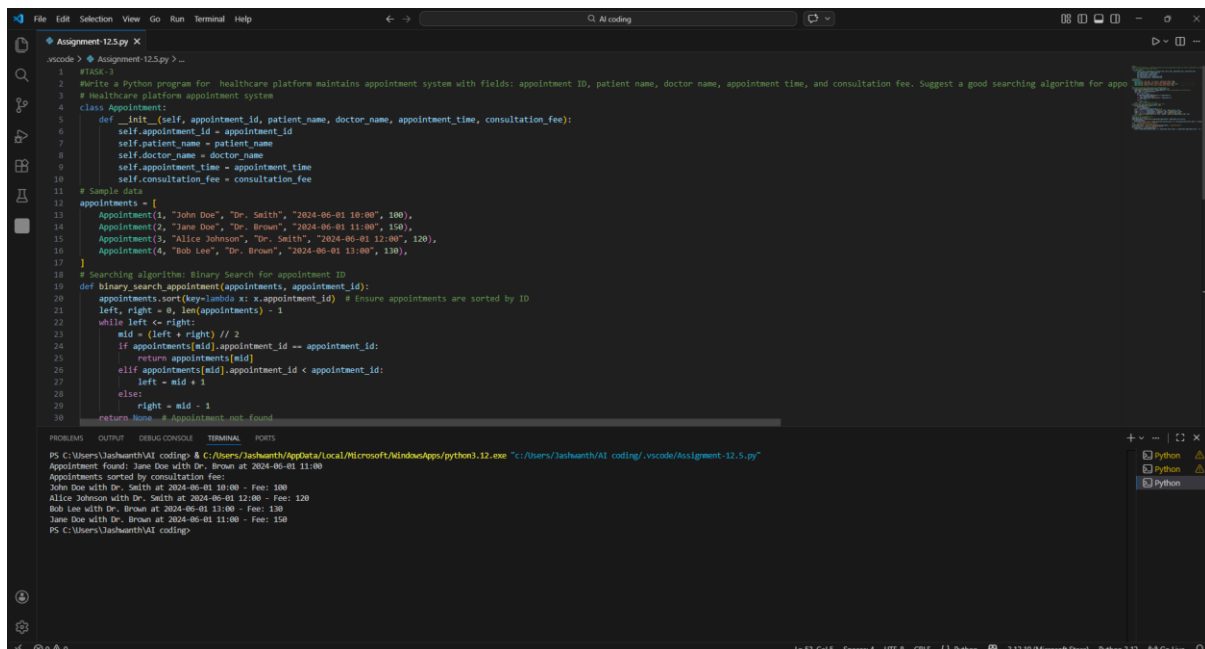
It was observed that Binary Search performs faster than linear search because it reduces the search space at each step, giving a time complexity of **$O(\log n)$** .

TASK-3:

PROMPT:

#Write a Python program for healthcare platform maintains appointment system with fields: appointment ID, patient name, doctor name, appointment time, and consultation fee. Suggest a good searching algorithm for appointment ID and a sorting algorithm for time or consultation fee, and implement the algorithm in Python.

CODE:



```
1 #TASK-1
2 #Write a Python program for healthcare platform maintains appointment system with fields: appointment ID, patient name, doctor name, appointment time, and consultation fee. Suggest a good searching algorithm for appo
3 # Healthcare platform appointment system
4 class Appointment:
5     def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
6         self.appointment_id = appointment_id
7         self.patient_name = patient_name
8         self.doctor_name = doctor_name
9         self.appointment_time = appointment_time
10        self.consultation_fee = consultation_fee
11
12    # Sample data
13    appointments = [
14        Appointment(1, "John Doe", "Dr. Smith", "2024-06-01 10:00", 100),
15        Appointment(2, "Jane Doe", "Dr. Brown", "2024-06-01 11:00", 150),
16        Appointment(3, "Alice Johnson", "Dr. Smith", "2024-06-01 12:00", 120),
17        Appointment(4, "Bob Lee", "Dr. Brown", "2024-06-01 13:00", 130),
18    ]
19
20    # Searching algorithm: Binary Search for appointment ID
21    def binary_search_appointment(appointments, appointment_id):
22        appointments.sort(key=lambda x: x.appointment_id) # Ensure appointments are sorted by ID
23        left, right = 0, len(appointments) - 1
24        while left <= right:
25            mid = (left + right) // 2
26            if appointments[mid].appointment_id == appointment_id:
27                return appointments[mid]
28            elif appointments[mid].appointment_id < appointment_id:
29                left = mid + 1
30            else:
31                right = mid - 1
32        return None # Appointment not found
33
34    # Sorting algorithm: Quick Sort for appointment time or consultation fee
35    def quick_sort(appointments, key=lambda x: x.appointment_time):
36        if len(appointments) <= 1:
37            return appointments
38        pivot = appointments[0]
39        less = [x for x in appointments if x[key] < pivot[key]]
40        greater = [x for x in appointments if x[key] > pivot[key]]
41        return quick_sort(less, key) + [pivot] + quick_sort(greater, key)
42
43    # Main execution
44    if __name__ == "__main__":
45        # Search for an appointment
46        appointment_id = 2
47        found_appointment = binary_search_appointment(appointments, appointment_id)
48        if found_appointment:
49            print(f"Appointment found: {found_appointment}")
50        else:
51            print(f"Appointment not found: {appointment_id}")
52
53        # Sort appointments by consultation fee
54        sorted_appointments = quick_sort(appointments, key=lambda x: x.consultation_fee)
55        print("Appointments sorted by consultation fee:")
56        for appointment in sorted_appointments:
57            print(f"{appointment.patient_name} with Dr. {appointment.doctor_name} at {appointment.appointment_time} - Fee: {appointment.consultation_fee}")
58
59        # Sort appointments by appointment time
60        sorted_appointments = quick_sort(appointments, key=lambda x: x.appointment_time)
61        print("Appointments sorted by appointment time:")
62        for appointment in sorted_appointments:
63            print(f"{appointment.patient_name} with Dr. {appointment.doctor_name} at {appointment.appointment_time} - Fee: {appointment.consultation_fee}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Jashwanth\AI coding> python Assignment-12.5.py
Appointment found: Jane Doe with Dr. Brown at 2024-06-01 11:00
Appointments sorted by consultation fee:
John Doe with Dr. Smith at 2024-06-01 10:00 - Fee: 100
Alice Johnson with Dr. Smith at 2024-06-01 12:00 - Fee: 120
Bob Lee with Dr. Brown at 2024-06-01 13:00 - Fee: 130
Jane Doe with Dr. Brown at 2024-06-01 11:00 - Fee: 150
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

In this task, involved designing a **healthcare appointment management system** that stores details such as appointment ID, patient name, doctor name, appointment time, and consultation fee.

The prompt asked the AI to recommend a suitable **searching algorithm for appointment ID** and a **sorting algorithm for appointment time or consultation fee**.

The AI suggested using **Binary Search for searching** because appointment IDs are unique and can be stored in sorted order. It also suggested using **Quick Sort or Merge Sort for sorting records** based on time or fee.

The generated Python code created an **Appointment class**, stored sample appointment records, implemented a search function to find appointments by ID, and used sorting functions to arrange appointments by time or consultation fee.

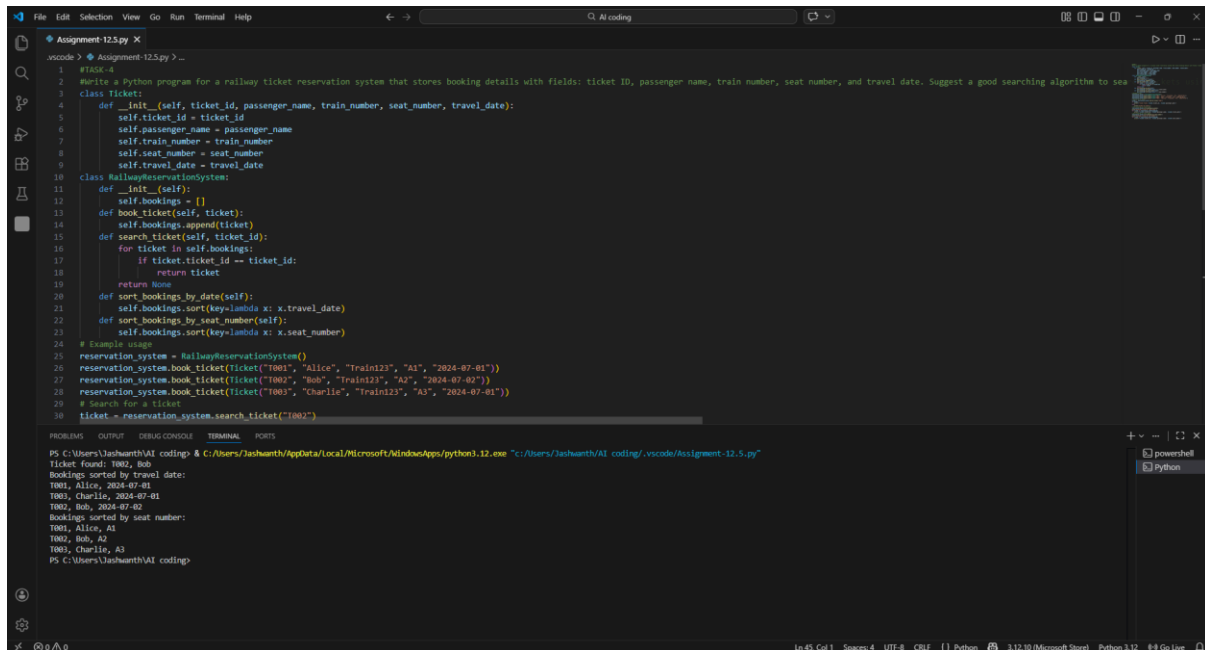
From the results, it was observed that searching allowed quick retrieval of appointment details, while sorting organized the records efficiently.

TASK-4:

PROMPT:

#Write a Python program for a railway reservation system that stores booking details with fields: ticket ID, passenger name, train number, seat number, and travel date. Suggest a good searching algorithm to search tickets using ticket ID and a sorting algorithm to sort bookings based on travel date or seat number, and implement the algorithms in Python.

CODE:



```
1 #TASK-4
2 Write a Python program for a railway ticket reservation system that stores booking details with fields: ticket ID, passenger name, train number, seat number, and travel date. Suggest a good searching algorithm to sea
3 class Ticket:
4     def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
5         self.ticket_id = ticket_id
6         self.passenger_name = passenger_name
7         self.train_number = train_number
8         self.seat_number = seat_number
9         self.travel_date = travel_date
10
11 class RailwayReservationSystem:
12     def __init__(self):
13         self.bookings = []
14     def book_ticket(self, ticket):
15         self.bookings.append(ticket)
16     def search_ticket(self, ticket_id):
17         for ticket in self.bookings:
18             if ticket.ticket_id == ticket_id:
19                 return ticket
20         return None
21     def sort_bookings_by_date(self):
22         self.bookings.sort(key=lambda x: x.travel_date)
23     def sort_bookings_by_seat_number(self):
24         self.bookings.sort(key=lambda x: x.seat_number)
25
26 # Example usage
27 reservation_system = RailwayReservationSystem()
28 reservation_system.book_ticket(Ticket("T001", "Alice", "Train123", "A1", "2024-07-01"))
29 reservation_system.book_ticket(Ticket("T002", "Bob", "Train123", "A2", "2024-07-02"))
30 reservation_system.book_ticket(Ticket("T003", "Charlie", "Train123", "A3", "2024-07-01"))
31 # Search for a ticket
32 ticket = reservation_system.search_ticket("T002")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\vscode\Assignment-12.5.py"
Ticket found: T002, Bob
Bookings sorted by travel date:
T001, Alice, 2024-07-01
T003, Charlie, 2024-07-01
T002, Bob, 2024-07-02
Bookings sorted by seat number:
T001, Alice, A1
T002, Bob, A2
T003, Charlie, A3
PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

In this task, a railway reservation system was created with fields such as **ticket ID**, **passenger name**, **train number**, **seat number**, and **travel date**.

The AI prompt asked for a searching algorithm to locate tickets using ticket ID and a sorting algorithm to arrange bookings by travel date or seat number.

The AI suggested using **Linear Search or Binary Search for ticket ID** and **sorting algorithms for organizing the records**. The generated code created a Ticket class and implemented functions to search and sort ticket data.

The results showed that searching correctly retrieved ticket information, while sorting arranged the tickets based on seat number or travel date.

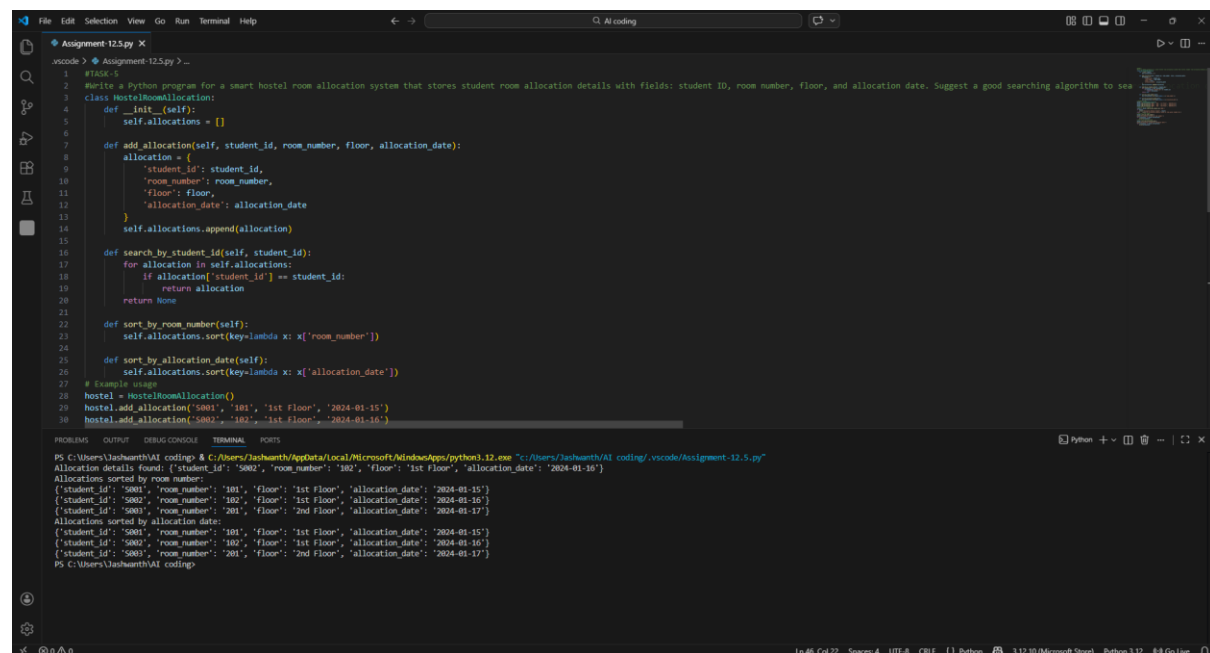
TASK-5:

PROMPT:

#Write a Python program for a smart hostel room allocation system that stores student room allocation details with fields: student ID, room number, floor, and allocation date. Suggest a good searching algorithm to search allocation details using student ID and a

sorting algorithm to sort records based on room number or allocation date, and implement the algorithms in Python.

CODE:



```
1 #Task-5
2 #Write a Python program for a smart hostel room allocation system that stores student room allocation details with fields: student ID, room number, floor, and allocation date. Suggest a good searching algorithm to sea
3
4 class HostelRoomAllocation:
5     def __init__(self):
6         self.allocations = []
7
8     def add_allocation(self, student_id, room_number, floor, allocation_date):
9         allocation = {
10             'student_id': student_id,
11             'room_number': room_number,
12             'floor': floor,
13             'allocation_date': allocation_date
14         }
15         self.allocations.append(allocation)
16
17     def search_by_student_id(self, student_id):
18         for allocation in self.allocations:
19             if allocation['student_id'] == student_id:
20                 return allocation
21         return None
22
23     def sort_by_room_number(self):
24         self.allocations.sort(key=lambda x: x['room_number'])
25
26     def sort_by_allocation_date(self):
27         self.allocations.sort(key=lambda x: x['allocation_date'])
28
29 # Example usage
30 hostel = HostelRoomAllocation()
31 hostel.add_allocation('S001', '101', '1st Floor', '2024-01-15')
32 hostel.add_allocation('S002', '102', '1st Floor', '2024-01-16')
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

```
PS C:\Users\jashwanth\AI coding> python Assignment-12.5.py
Allocation details found: {'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}
Allocations sorted by room number:
[{'student_id': 'S001', 'room_number': '101', 'floor': '1st Floor', 'allocation_date': '2024-01-15'}, {'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}, {'student_id': 'S003', 'room_number': '201', 'floor': '2nd Floor', 'allocation_date': '2024-01-17'}]
Allocations sorted by allocation date:
[{'student_id': 'S001', 'room_number': '101', 'floor': '1st Floor', 'allocation_date': '2024-01-15'}, {'student_id': 'S002', 'room_number': '102', 'floor': '1st Floor', 'allocation_date': '2024-01-16'}, {'student_id': 'S003', 'room_number': '201', 'floor': '2nd Floor', 'allocation_date': '2024-01-17'}]
PS C:\Users\jashwanth\AI coding>
```

OBSERVATION:

In this task, it focused on managing **hostel room allocation data**, which includes student ID, room number, floor, and allocation date.

The AI prompt requested an optimized searching algorithm for student ID and a sorting algorithm for room number or allocation date.

The AI-generated code created a class to represent room allocation records and implemented functions to search student records and sort the allocation list.

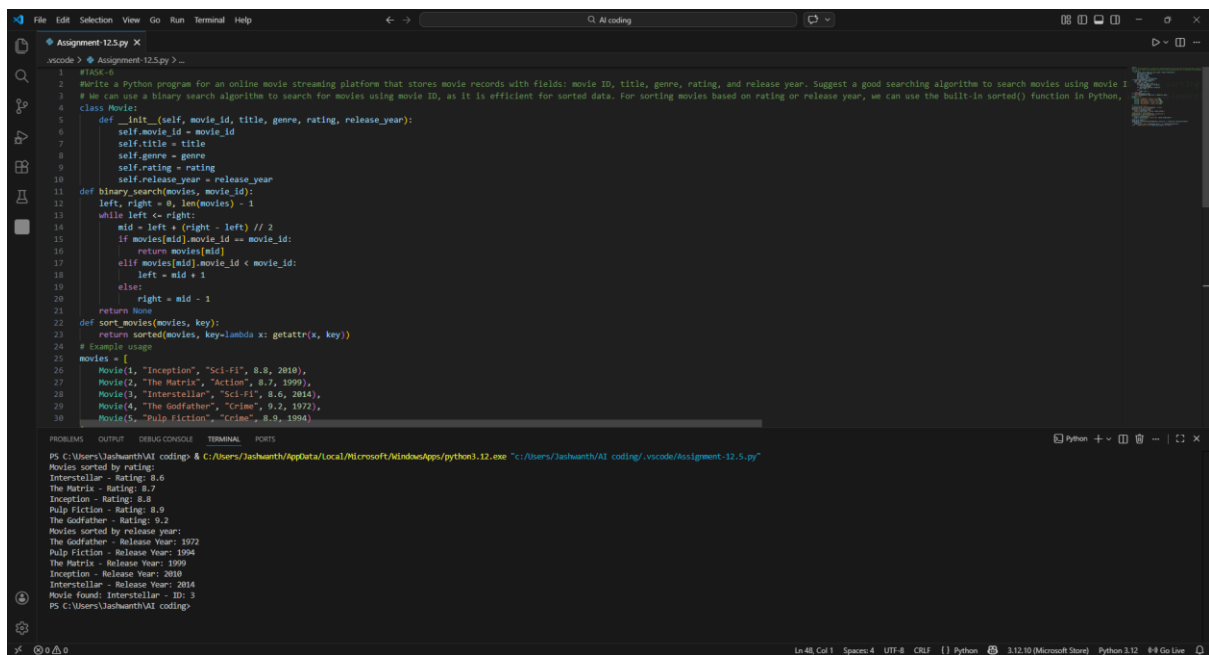
It was observed that sorting the records helped maintain organized hostel allocation data, and searching made it easy to locate a specific student's room information.

TASK-6:

PROMPT:

#Write a Python program for an online movie streaming platform that stores movie records with fields: movie ID, title, genre, rating, and release year. Suggest a good searching algorithm to search movies using movie ID and a sorting algorithm to sort movies based on rating or release year, and implement the algorithms in Python.

CODE:



```
1 # Write a Python program for an online movie streaming platform that stores movie records with fields: movie ID, title, genre, rating, and release year. Suggest a good searching algorithm to search movies using movie ID.
2 # We can use a binary search algorithm to search for movies using movie ID, as it is efficient for sorted data. For sorting movies based on rating or release year, we can use the built-in sorted() function in Python.
3
4 class Movie:
5     def __init__(self, movie_id, title, genre, rating, release_year):
6         self.movie_id = movie_id
7         self.title = title
8         self.genre = genre
9         self.rating = rating
10        self.release_year = release_year
11
12    def binary_search(movies, movie_id):
13        left, right = 0, len(movies) - 1
14        while left <= right:
15            mid = left + (right - left) // 2
16            if movies[mid].movie_id == movie_id:
17                return movies[mid]
18            elif movies[mid].movie_id < movie_id:
19                left = mid + 1
20            else:
21                right = mid - 1
22        return None
23
24    def sort_movies(movies, key):
25        return sorted(movies, key=lambda x: getattr(x, key))
26
27 # Example usage
28 movies = [
29     Movie(1, "Inception", "Sci-Fi", 8.8, 2010),
30     Movie(2, "The Matrix", "Action", 8.7, 1999),
31     Movie(3, "Interstellar", "Sci-Fi", 8.6, 2014),
32     Movie(4, "The Godfather", "Crime", 9.2, 1972),
33     Movie(5, "Pulp Fiction", "Crime", 8.9, 1994)
34 ]
35
36 # Search for a movie by ID
37 movie_id = 3
38 result = Movie.binary_search(movies, movie_id)
39 if result:
40     print(f"Movie found: {result.title} (ID: {result.movie_id})")
41 else:
42     print("Movie not found.")
43
44 # Sort movies by rating
45 sorted_by_rating = Movie.sort_movies(movies, 'rating')
46 print("Movies sorted by rating:")
47 for movie in sorted_by_rating:
48     print(f"{movie.title} - Rating: {movie.rating}")
49
50 # Sort movies by release year
51 sorted_by_release_year = Movie.sort_movies(movies, 'release_year')
52 print("Movies sorted by release year:")
53 for movie in sorted_by_release_year:
54     print(f"{movie.title} - Release Year: {movie.release_year}")
```

PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\Jashwanth\AI coding\vscode\Assignment-12.5.py"

Movies sorted by rating:
Interstellar - Rating: 8.6
The Matrix - Rating: 8.7
Inception - Rating: 8.8
Pulp Fiction - Rating: 8.9
The Godfather - Rating: 9.2
Movies sorted by release year:
The Godfather - Release Year: 1972
Pulp Fiction - Release Year: 1994
The Matrix - Release Year: 1999
Inception - Release Year: 2010
Interstellar - Release Year: 2014
Movie found: Interstellar - ID: 3
PS C:\Users\Jashwanth\AI coding>

OBSERVATION:

In this task, it simulate a **movie database system** with fields such as movie ID, title, genre, rating, and release year.

The prompt asked the AI to suggest suitable algorithms for searching movies using movie ID and sorting movies based on rating or release year.

The generated code stored movie records and implemented search and sorting functions. The program successfully retrieved movie information and displayed sorted movie lists.

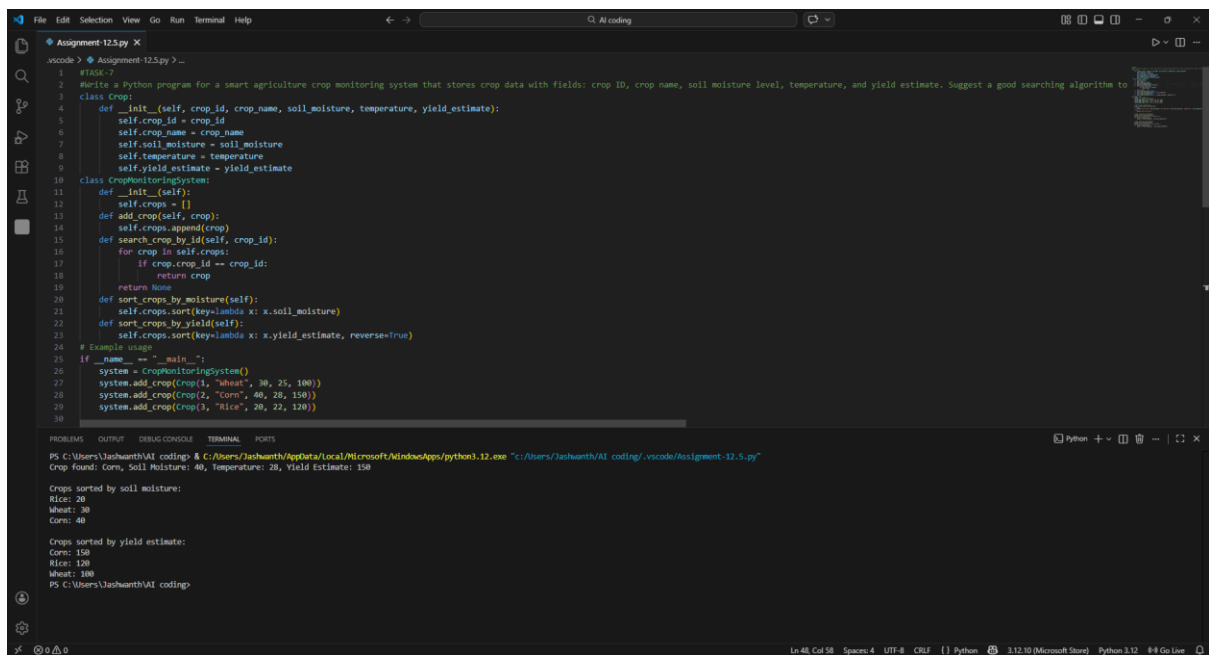
This demonstrated how sorting helps users easily view movies based on ratings or release year.

TASK-7:

PROMPT:

#Write a Python program for a smart agriculture crop monitoring system that stores crop data with fields: crop ID, crop name, soil moisture level, temperature, and yield estimate. Suggest a good searching algorithm to search crop details using crop ID and a sorting algorithm to sort crops based on soil moisture level or yield estimate, and implement the algorithms in Python.

CODE:



```
1 #task-7
2 #Write a Python program for a smart agriculture crop monitoring system that stores crop data with fields: crop ID, crop name, soil moisture level, temperature, and yield estimate. Suggest a good searching algorithm to
3 class Crop:
4     def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
5         self.crop_id = crop_id
6         self.crop_name = crop_name
7         self.soil_moisture = soil_moisture
8         self.temperature = temperature
9         self.yield_estimate = yield_estimate
10 class CropMonitoringSystem:
11     def __init__(self):
12         self.crops = []
13     def add_crop(self, crop):
14         self.crops.append(crop)
15     def search_crop_by_id(self, crop_id):
16         for crop in self.crops:
17             if crop.crop_id == crop_id:
18                 return crop
19         return None
20     def sort_crops_by_moisture(self):
21         self.crops.sort(key=lambda x: x.soil_moisture)
22     def sort_crops_by_yield(self):
23         self.crops.sort(key=lambda x: x.yield_estimate, reverse=True)
24 # Example usage
25 if __name__ == "__main__":
26     system = CropMonitoringSystem()
27     system.add_crop(crop(1, "Wheat", 30, 25, 100))
28     system.add_crop(crop(2, "Corn", 40, 28, 150))
29     system.add_crop(crop(3, "Rice", 20, 22, 120))
30
31 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
32 PS C:\Users\Jashwanth\AI coding> & C:\Users\Jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:\Users\Jashwanth\AI coding\vscode\Assignment-12.5.py"
33 Crop found: Corn, Soil Moisture: 40, Temperature: 28, Yield Estimate: 150
34
35 Crops sorted by soil moisture:
36 Rice: 20
37 Wheat: 30
38 Corn: 40
39
40 Crops sorted by yield estimate:
41 Corn: 150
42 Rice: 120
43 Wheat: 100
44 PS C:\Users\Jashwanth\AI coding>
```

OBSERVATION:

In this task, it involved managing crop data including **crop ID, crop name, soil moisture level, temperature, and yield estimate**.

The prompt requested a searching algorithm for crop ID and a sorting algorithm for moisture level or yield estimate.

The AI-generated code created a Crop class and implemented search and sorting functions. When executed, the program successfully retrieved crop details and sorted crops based on moisture or yield values.

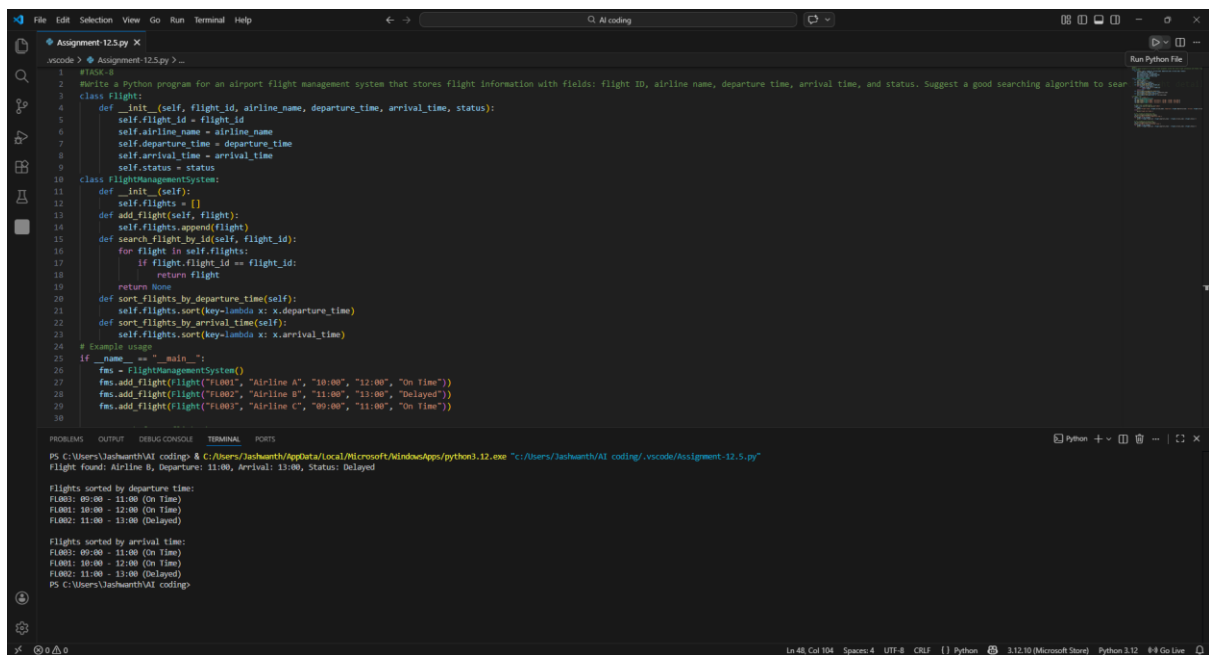
This demonstrated how algorithm-based data management can assist farmers in monitoring crop conditions efficiently.

TASK-8:

PROMPT:

#Write a Python program for an airport flight management system that stores flight information with fields: flight ID, airline name, departure time, arrival time, and status. Suggest a good searching algorithm to search flight details using flight ID and a sorting algorithm to sort flights based on departure time or arrival time, and implement the algorithms in Python.

CODE:



```
1 # Task 9
2 Write a Python program for an airport flight management system that stores flight information with fields: flight ID, airline name, departure time, arrival time, and status. Suggest a good searching algorithm to search for a flight by ID.
3 class Flight:
4     def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
5         self.flight_id = flight_id
6         self.airline_name = airline_name
7         self.departure_time = departure_time
8         self.arrival_time = arrival_time
9         self.status = status
10 class FlightManagementSystem:
11     def __init__(self):
12         self.flights = []
13     def add_flight(self, flight):
14         self.flights.append(flight)
15     def search_flight_by_id(self, flight_id):
16         for flight in self.flights:
17             if flight.flight_id == flight_id:
18                 return flight
19         return None
20     def sort_flights_by_departure_time(self):
21         self.flights.sort(key=lambda x: x.departure_time)
22     def sort_flights_by_arrival_time(self):
23         self.flights.sort(key=lambda x: x.arrival_time)
24 # Example usage
25 if __name__ == "__main__":
26     fms = FlightManagementSystem()
27     fms.add_flight(flight("FL001", "Airline A", "10:00", "12:00", "On Time"))
28     fms.add_flight(flight("FL002", "Airline B", "11:00", "13:00", "Delayed"))
29     fms.add_flight(flight("FL003", "Airline C", "09:00", "11:00", "On Time"))
30
31 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
32 PS C:\Users\jashwanth\AI coding> & C:\Users\jashwanth\AppData\Local\Microsoft\WindowsApps\python3.12.exe "C:\Users\jashwanth\AI coding\.vscode\Assignment-12.5.py"
33 Flight found: Airline B, Departure: 11:00, Arrival: 13:00, Status: Delayed
34
35 Flights sorted by departure time:
36 FL003: 09:00 - 11:00 (On Time)
37 FL001: 10:00 - 12:00 (On Time)
38 FL002: 11:00 - 13:00 (Delayed)
39
40 Flights sorted by arrival time:
41 FL003: 09:00 - 11:00 (On Time)
42 FL001: 10:00 - 12:00 (On Time)
43 FL002: 11:00 - 13:00 (Delayed)
44 PS C:\Users\jashwanth\AI coding>
```

OBSERVATION:

In this task, was to develop a system that stores flight details such as **flight ID, airline name, departure time, arrival time, and status**.

The prompt asked the AI to recommend algorithms for searching flight details using flight ID and sorting flights based on departure or arrival time.

The generated code created a Flight class and implemented search and sorting functions. The program successfully displayed sorted flight schedules and retrieved specific flight information.

This task showed how searching and sorting algorithms help manage transportation data effectively.